

慶應義塾大学 理工学部 情報工学科 自然言語処理 (2026/06/18)

10 - 言語モデルを転移学習する

情報工学科 准教授 高道 慎之介

講義予定

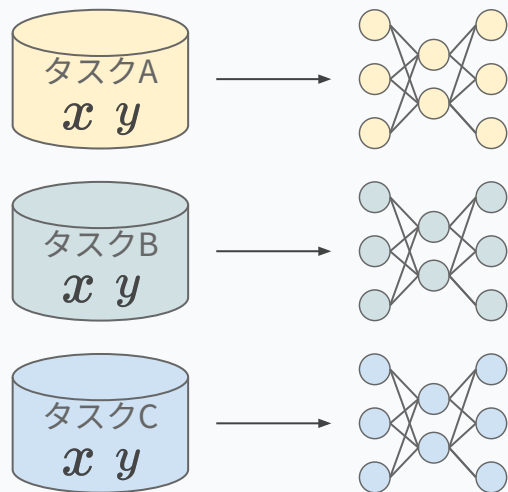
第XX回	日付	内容（順次変わっていくので予想）	
第01回	2026/04/09	イントロダクション	事前知識の確認
第02回	2026/04/16	自然言語処理のための機械学習に入門する	
第03回	2026/04/23	言語学に入門する	言語とは何だろう
第04回	2026/04/30	単語をベクトルで表現する	自然言語処理の基礎技術
第05回	2026/05/07	言語モデルで文章を統計的に扱う	
第06回	2026/05/14	系列にラベリングする	
第07回	2026/05/21	演習 1	これまでの復習
第08回	2026/05/28	Transformer を理解する	大規模言語モデル時代の技術
第09回	2026/06/11	言語モデルを事前学習する	
第10回	2026/06/18	言語モデルを転移学習する	
第11回	2026/06/25	言語を生成 (デコーディング) する	
第12回	2026/07/02	言語やラベルを評価する	
第13回	2026/07/09	自然言語処理から音声言語処理へ	おまけ
第14回	2026/07/16	演習 2	これまでの復習
期末試験	2026/07/XX	(日程は後日アナウンス)	

復習：事前学習

伝統的な機械学習 vs. 最近の機械学習

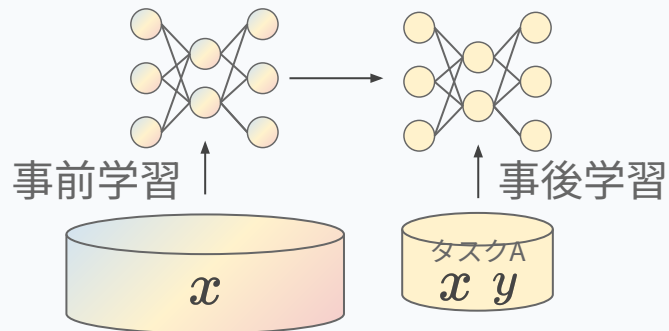
伝統的な機械学習

1. データ (x, y) のペアを大量に用意
2. モデル $f()$, $g()$ を乱数で初期化
3. (x, y) を使って $f()$, $g()$ を学習
 - a. supervised learning



最近の機械学習

1. x のみ (or y のみ) を大量に用意
2. モデル $f()$, $g()$ を乱数で初期化
3. x (or y) を使って $f()$, $g()$ を学習
4. データ (x, y) のペアを少量だけ用意
5. (x, y) を使って $f()$, $g()$ を微調整
 - a. “supervised fine-tuning (SFT)”



x から y の予測

y

$d(\cdot)$

決定 (decision)

- 出力候補のうちから 1 つを決定すること

$g(\cdot)$

スコア計算 (score calculation)

- 特徴に基づいて、出力候補とそのスコアを計算すること

$f(\cdot)$

特徴抽出 (feature extraction)

- 意味のある特徴を入力テキストから抽出すること

x

この 2 つを x のみで予め学習することが事前学習の意味

“事前” 学習 (pre-training) の意味

大量のラベル無しデータ (テキストのみ) を学習に利用できる

ニューラルネットワークのスケラビリティの恩恵を受けられる

少量のラベル付きデータを準備するだけで良い

データ収集と学習に関するコストを抑えられる

広範な NLP タスクに転用できる特徴抽出・スコア計算を学習可能

ラベル無しデータだけで事前学習して「良い」パラメータを事前に推定可能

どれくらい大規模なデータ？

最近のモデルの事前学習データ

	トークン数 [兆]
gpt-oss120B	数兆
Llama3.1	15~
DeepSeek-v3	14.8
Qwen3	36
GPT-3	0.5
OLMo 2	3.9

LLM 大規模言語モデル講座 講義資料 © 2025 (CC BY-NC-ND 4.0)

LLM .jp v4 モデルの事前学習データ

サブワードが平均 2文字/トークンなら、
24兆文字相当 10万文字/冊とするなら2.4億冊相当？

全体の学習コーパス量 (合計 11.74T tokens)

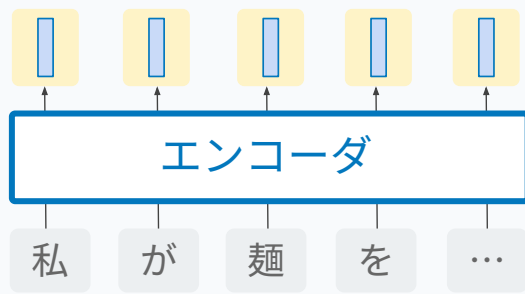
- Pre-training Phase1 6.29T tokens
(4096x512x3Msteps / 4096x1024x1.5M steps)
- Pre-training Phase2 4.19T tokens
(4096 x 2048x500K steps)
- Mid-training Phase1 0.84T tokens
(4096 x 2048 x 100K steps)
- Mid-training Phase2 0.21T tokens
(16K x 512 x 25K steps)
- Mid-training Phase3 0.21T tokens
(64K x 128 x 25K steps)

<https://llm-jp.nii.ac.jp/resources/> (2026-05-19)

大規模言語モデルアーキテクチャの型

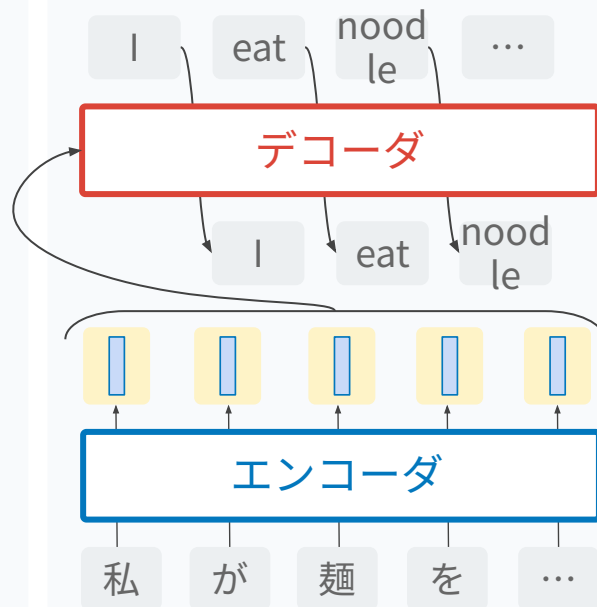
Encoder-only 型

BERT, RoBERTa, etc.



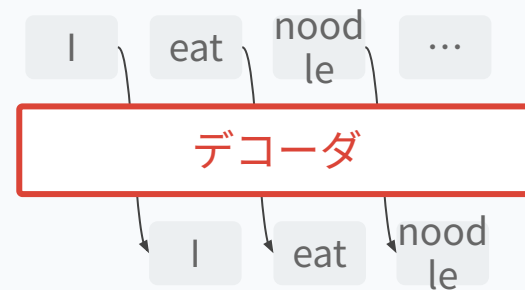
Encoder-decoder 型

BART, T5, etc.



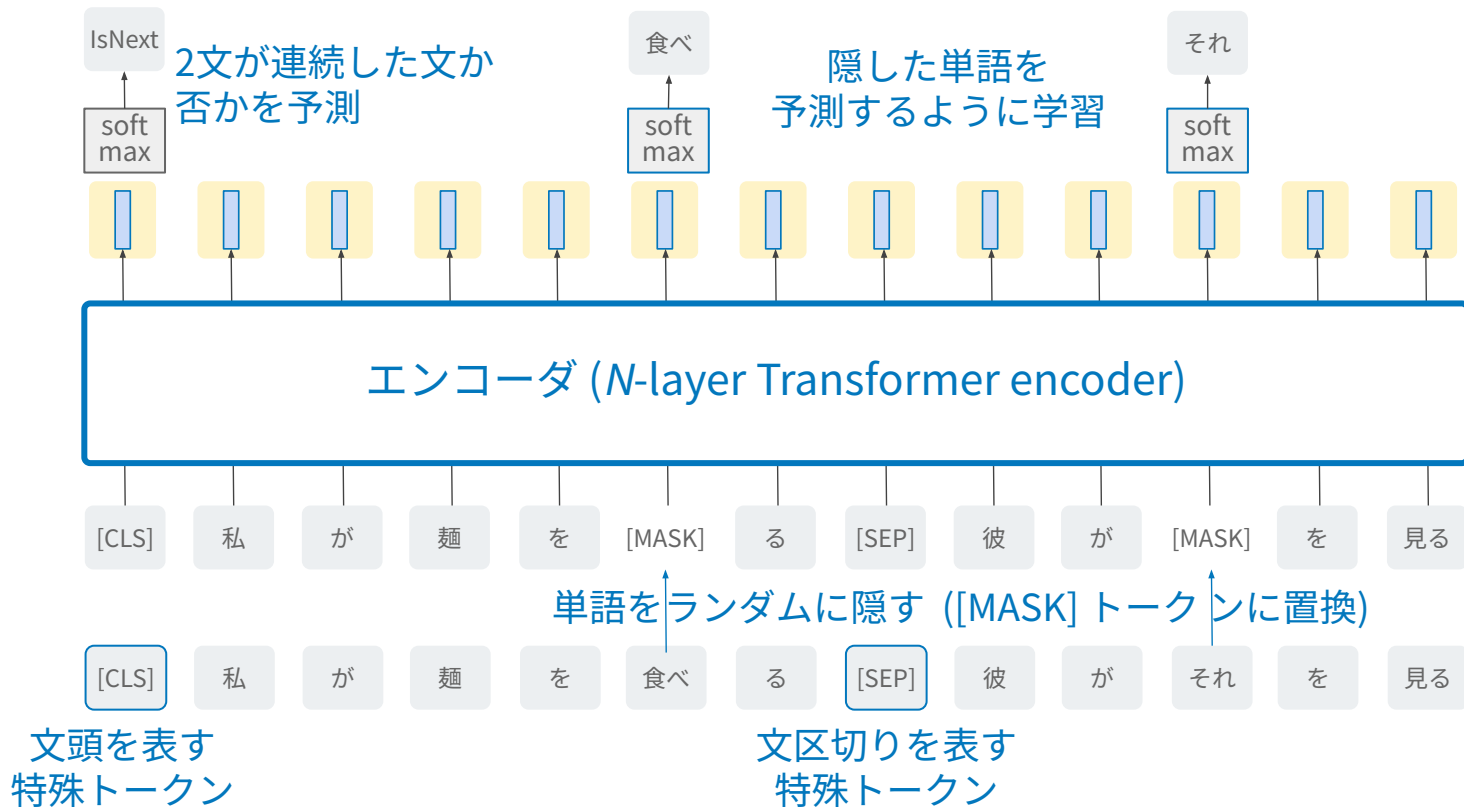
Decoder-only 型

GPT, LLaMA, etc.

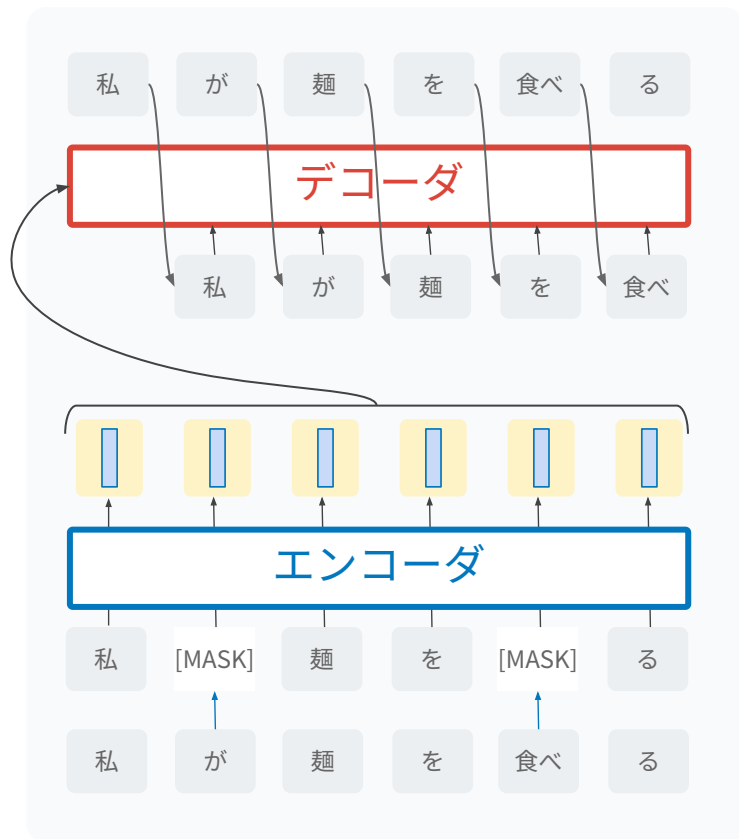


それぞれの型をどうやって事前学習するのかを知ろう

BERT：マスク言語モデルに基づく事前学習



BART: ディノイズングオートエンコーダに基づく事前学習



Encoder-only 型と比べて

- 特徴量抽出 (encoder) だけでなくテキスト生成 (decoder) も事前学習可能

Decoder-only 型と比べて

- 全単語の関係から特徴を抽出可能 (decoder-only は過去の単語のみ)

Denoising の意味

- 壊れた文から元の文を復元するように学習することは、抽象的要約と同じ効果

Decoder-only 型の事前学習



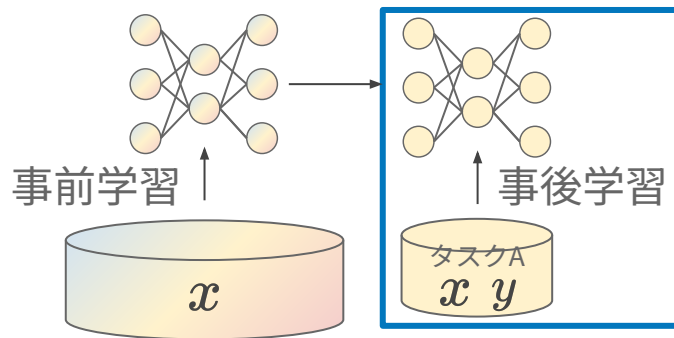
次トークン予測 (next token prediction)で学習

- t 番目の単語を，それ以前の単語から予測

Encoder-decoder 型と比べて

- KV キャッシュによる計算高速化が可能 (課題)
- 推論コストが小さい
 - encoder は self-attention で双方向計算を行うため重い
 - decoder は過去トークンのみ
- 長文に対する推論コストが小さい

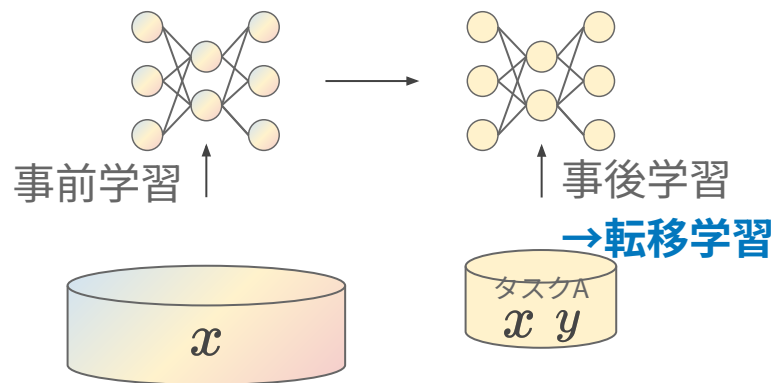
今日の目的： 大規模言語モデルを転移学習する



転移学習 (≡ fine-tuning) の事例

転移学習とは

新しいタスクに対する有効な仮説を効率的に見つけ出すために、
一つ以上の別のタスクで学習された知識を保持・適用する問題を指す。



数百～数千例

	トークン数 [兆]
gpt-oss120B	数兆
Llama3.1	15~
DeepSeek-v3	14.8
Qwen3	36
GPT-3	0.5
OLMo 2	3.9

大規模言語モデルの隆盛

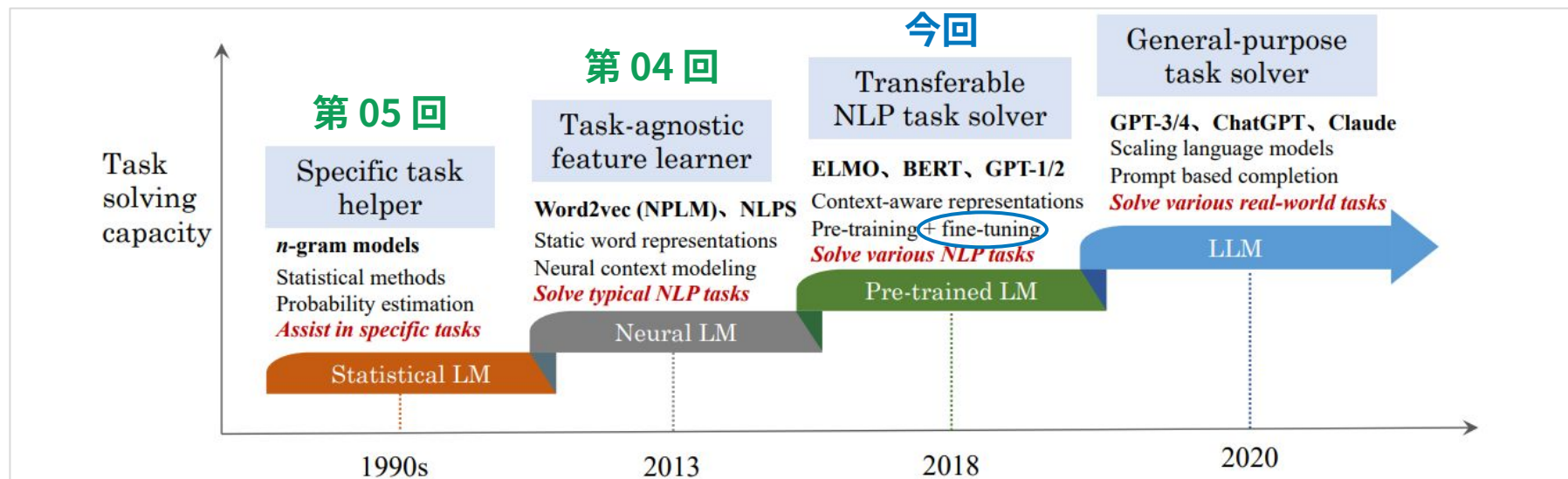


Fig. 2: An evolution process of the four generations of language models (LM) from the perspective of task solving capacity. Note that the time period for each stage may not be very accurate, and we set the time mainly according to the publish date of the most representative studies at each stage. For neural language models, we abbreviate the paper titles of two representative studies to name the two approaches: NPLM [1] (“A neural probabilistic language model”) and NLPS [2] (“Natural language processing (almost) from scratch”). Due to the space limitation, we don’t list all representative studies in this figure.

各社による, fine-tuning 済み (マルチモーダル) LLM

Med-Gemini (Google)

[Home](#) > [Blog](#) >

Advancing medical AI with Med-Gemini

May 15, 2024 ·

Greg Corrado, Distinguished Scientist, Google Research, and Joëlle Barral, Senior Director, Google DeepMind



An introduction to Med-Gemini, a family of Gemini models fine-tuned for multimodal medical domain applications.

QUICK LINKS

- Capabilities of Gemini Models in Medicine
- Advancing Multimodal Medical Capabilities...

Share

For AI models to perform well on diverse medical tasks and to meaningfully assist

<https://research.google/blog/advancing-medical-ai-with-med-gemini/>

PLaMo Fin Prime (PFN)

2025.06.17 Tech

PLaMo Fin Primeをリリースしました

Area [Machine Learning / Deep Learning](#) [Natural Language Processing / Speech Processing](#) [Quantitative Finance](#)

Tag [LLM](#) [GPT](#) [金融](#)



Masanori Hirano

Preferred Networks (PFN) は、金融知識を強化した大規模言語モデル (LLM) である「PLaMo Fin Prime」をリリースしました。このモデルは、日本語能力の高いPLaMoをベースとして、金融に特化したコーパスを追加学習することで構築が行われており、日本語能力の高さと金融能力の高さを兼ね備えたドメインに特化したモデルとなっています。

また、今回の開発ノウハウを生かして、個社独自のデータを用いた専用モデルを構築するサービスについても提供開始をしました。

今回の記事では、この詳細について紹介します。

LLM分野におけるPLaMo Fin Primeの位置づけ

PLaMoはPreferred Networksグループが開発している大規模言語モデル(LLM)です。

一般的なLLMの構築手法ではまず初めに、非常に多くの文書を学習させて文の続きを書くという能力を身につけた事前学習モデルを作成します。この事前学習の過程で、LLMは様々な知識を身につけます。そのうえで、ユーザーの質問にうまく回答したり、安全性を高めるような、事後学習(アライメント学習)を行うのが一般的です。

すでに公開されている、一般向けのPLaMo 1.0 Primeや2.0 Primeは、この事前学習を汎用コーパス(文書)で実施したのちに、汎用向けの事後学習を行ったモデルとなっています。

では、このモデルをドメインに特化させる、つまり、金融に特化させるとなった時に、どのような取り組みを行うべきかということについて考えてみます。

一般に、「金融に特化する」といったときに、最も重要になるのは金融知識ではないかと思えます。

<https://tech.preferred.jp/ja/blog/20250617-plamo-fin-prime-release/>

Codex (OpenAI)

Abstract

We introduce Codex, a GPT language model fine-tuned on publicly available code from GitHub, and study its Python code-writing capabilities. A distinct production version of Codex powers GitHub Copilot. On [HumanEval](#), a new evaluation set we release to measure functional correctness for synthesizing programs from docstrings, our model solves 28.8% of the problems, while GPT-3 solves 0% and GPT-J solves 11.4%. Furthermore, we find that repeated sampling from the model is a surprisingly effective strategy for producing working solutions to difficult prompts. Using this method, we solve 70.2% of our problems with 100 samples per problem. Careful investigation of our model reveals limitations, including

<https://arxiv.org/pdf/2107.03374>

API を介した、ユーザ自身による fine-tuning

Gemini (Google)

教師ありファイン チューニングを使用して Gemini モデルをチューニングする

フィードバックを送信



このページの内容 ▼

始める前に

サポートされているモデル

チューニング ジョブを作成する

ハイパーパラメータのチューニング

チューニング ジョブのリストを表示する

...

このドキュメントでは、教師ありファインチューニングを使用して Gemini モデルをチューニングする方法について説明します。

<https://docs.cloud.google.com/gemini-enterprise-agent-platform/models/tuning/supervised-tuning/use?hl=ja>

GPT-4o (OpenAI) (まもなく終了)

GPT-4o でファインチューニングが 利用できるようになりました

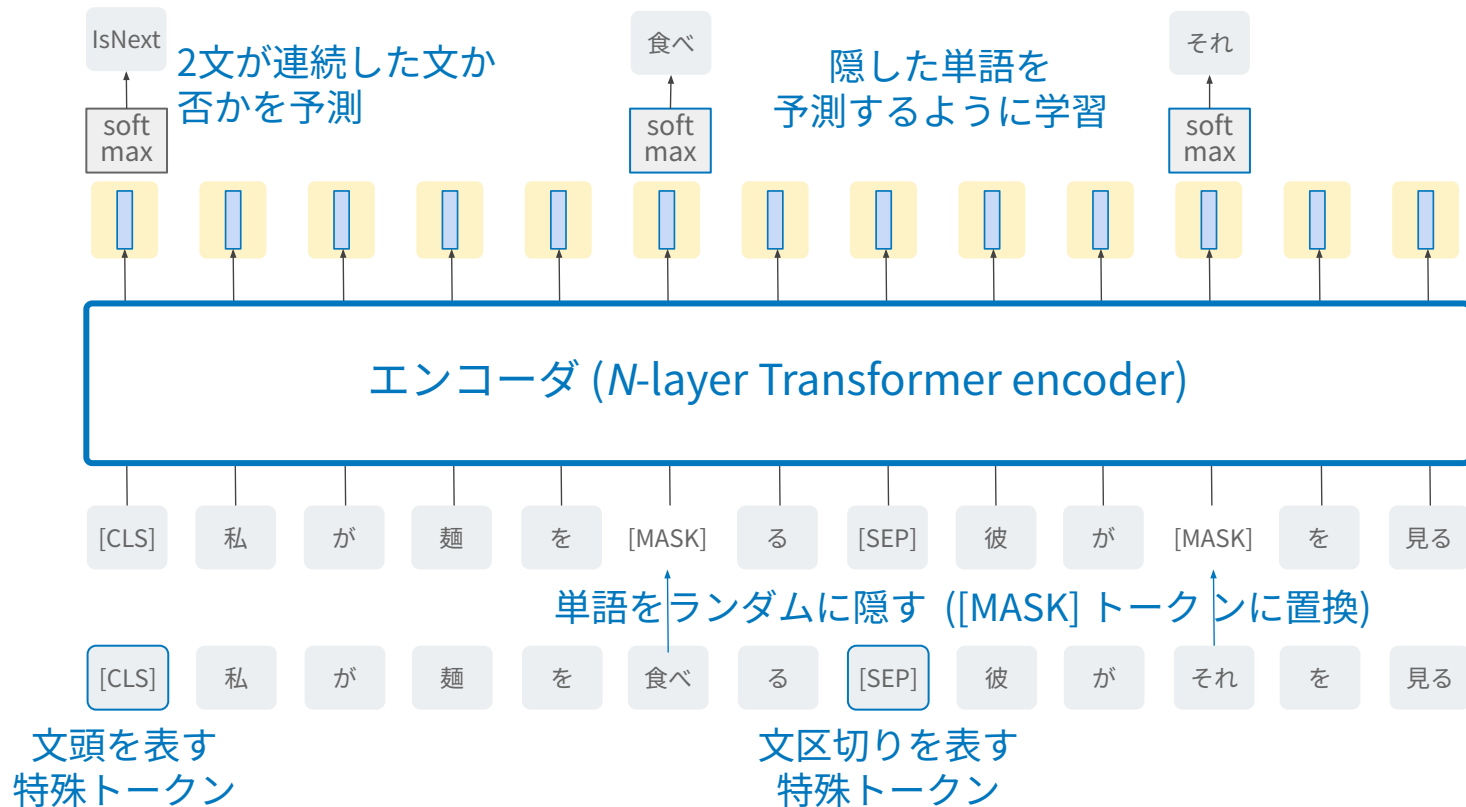
GPT-4o のカスタムバージョンをファインチューニングして、
アプリケーションのパフォーマンスと精度を向上させます。



<https://openai.com/ja-JP/index/gpt-4o-fine-tuning/>

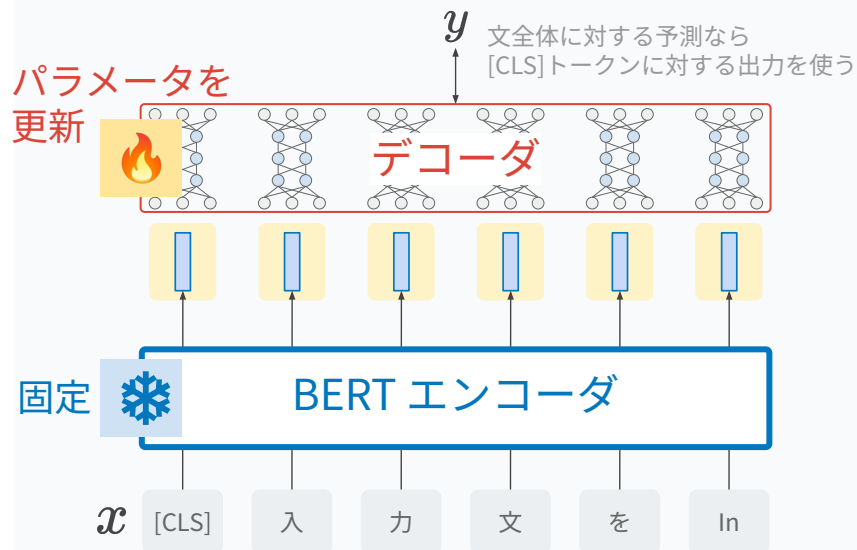
従来の (≡ BERT 時代の) 転移学習

BERT：マスク言語モデルに基づく事前学習

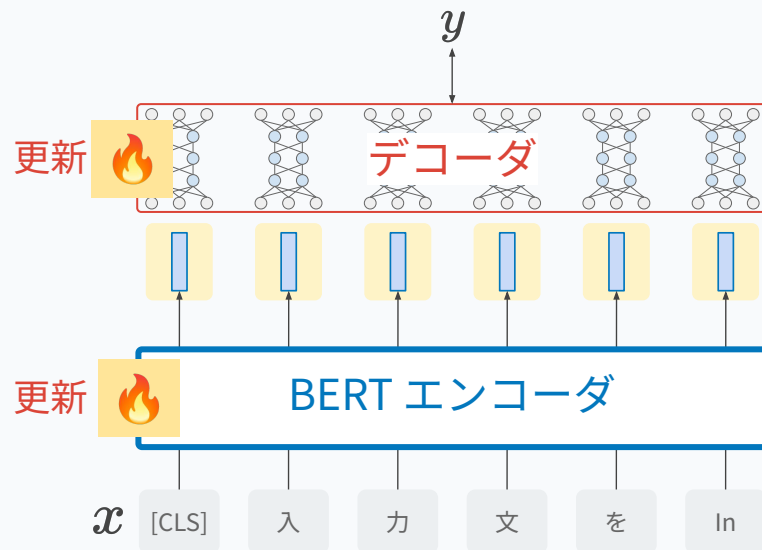


事前学習済み BERT を用いた転移学習

学習済み特徴量抽出器として使用



抽出器も追加学習 (full finetuning)



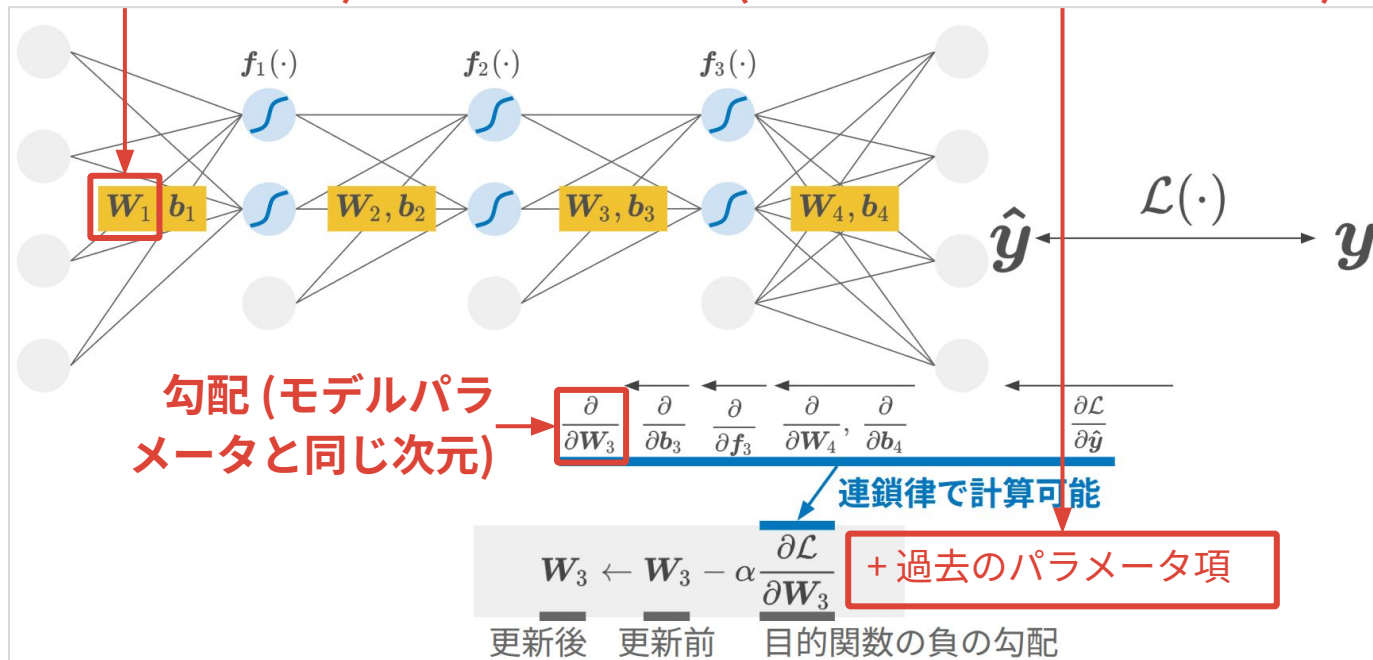
一方で LLM はモデルサイズが超巨大 → 学習のための (GPU) メモリが必要

事前知識：モデル学習に必要なメモリ

実際にはミニバッチサイズなどでメモリ数は変わるので注意

モデルパラメータ
(N-by-N 行列なら N^2)

Adam などは2次勾配を使う
(モデルパラメータと同じ次元)



自然言語処理 第02回資料から引用

各パラメータに対し浮動小数点 (float16 or float32) のメモリが必要

例えばどれくらいの GPU メモリが必要？

gpt-oss が登場

オープンウェイトリゾニングモデルの限界を
押し広げる gpt-oss-120b と gpt-oss-20b

パラメータ数が 20×10^9

[Hugging Face の詳細を見る](#)

[System Card を読む](#)

(BはBillion [10億])

▶ 記事を音声で聴く 16:42

共有

はじめに

低コストで強力な実世界パフォーマンスを実現する2つの最先端のオープンウェイト言語モデルの gpt-oss-120b と gpt-oss-20b をリリースします。Apache 2.0 ライセンスの下で利用可能なこれらのモデルは、推論タスクにおいて同規模の open model よりも優れたパフォーマンスを発揮し、強力なツール使用機能を備えており、一般的に広く使用されているハードウェア上での効率的な展開のために最適化されています。また、強化学習と o3 その他のフロンティアシステムを含む OpenAI の最先端の内部モデルに基づく手法を組み合わせて学習しています。

<https://openai.com/ja-JP/index/introducing-gpt-oss/>

要素	計算式	20Bモデルの理論値
モデル	$N_{\text{param}} \times \text{size}(\text{float16})$	40GB
勾配	$N_{\text{param}} \times \text{size}(\text{float16})$	40GB
最適化器 (Adam)	$N_{\text{param}} \times \text{size}(\text{float16}) \times 2$	80GB
小計	$N_{\text{param}} \times \text{size}(\text{float16}) \times 4$	160GB



NVIDIA H200 NVL 141GB [PCIExp 141GB]

最安価格
5,305,000円

メーカー希望小売価格: -円

価格推移グラフ

売れ筋ランキング
👑 92 位 (15917製品中)

レビュー
📝 レビューを書く

クチコミ
📝 クチコミを書く

お気に入り
♡ お気に入り (7人)

スペック・仕様
[すべてのスペック・仕様を見る](#)

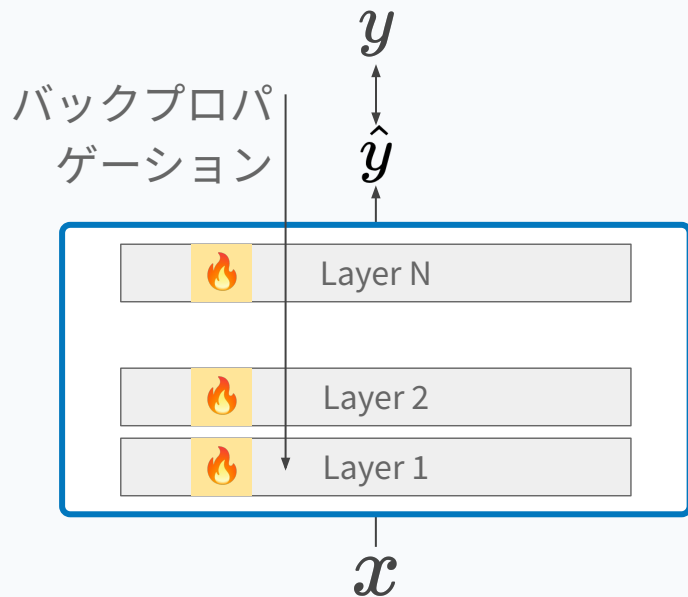
画像提供: アプライドネット

Parameter-efficient fine-tuning (PEFT)

Full fine-tuning vs. Parameter-efficient fine-tuning

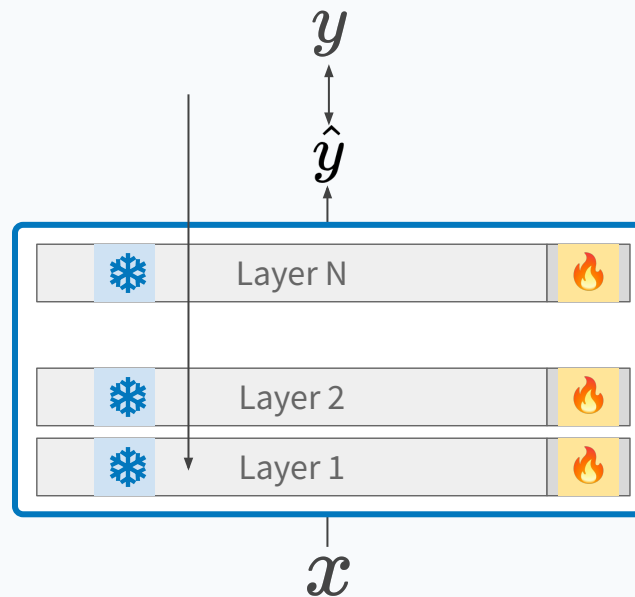
Full fine-tuning

事前学習済みモデルのすべてのパラメータを更新する



Parameter-efficient fine-tuning

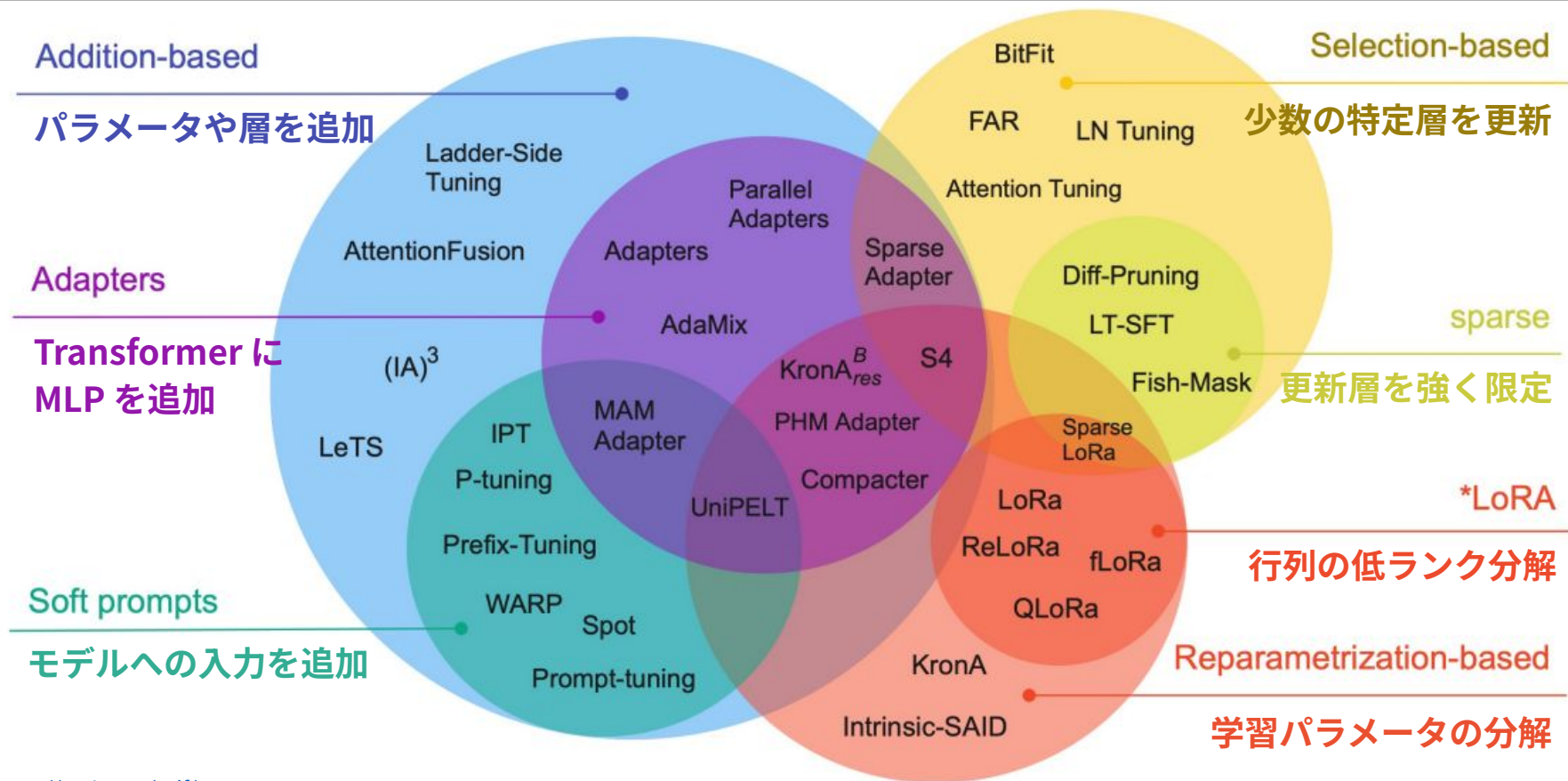
追加したパラメータあるいは元のモデルの一部パラメータのみを更新する



Full fine-tuning vs. Parameter-efficient fine-tuning

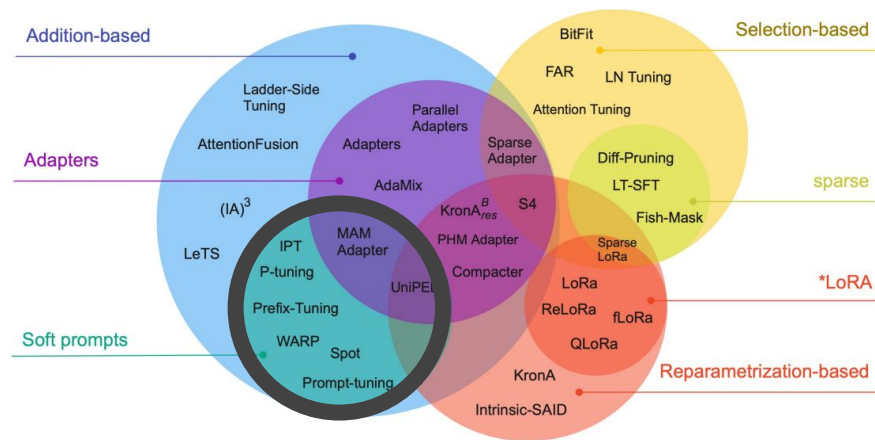
要素	Full fine-tuning (Full FT)	Parameter-efficient fine-tuning (PEFT)
モデル	$N_{\text{param}} \times \text{size}(\text{float16}) \rightarrow 40\text{GB}$	$N_{\text{param}} \times \text{size}(\text{float16}) \rightarrow 40\text{GB}$
勾配	$N_{\text{param}} \times \text{size}(\text{float16}) \rightarrow 40\text{GB}$	$N_{\text{train}} \times \text{size}(\text{float16}) \rightarrow \text{数十MB}$
最適化器	モデルパラメータ数 (例えば 10^9) $N_{\text{param}} \times \text{size}(\text{float16}) \times 2 \rightarrow 80\text{GB}$	学習するパラメータ数 (例えば 10^5) $N_{\text{train}} \times \text{size}(\text{float16}) \times 2 \rightarrow \text{数十MB}$
ストレージ	元のモデル (N_{param}) + FT済みモデル (N_{param}) $\rightarrow 80\text{GB}$	元のモデル (N_{param}) + 追加モデル (N_{train}) $\rightarrow 40\text{GB} + \alpha$
性能	一般的に良い	FTよりは悪い
必要データ	多くのデータが必要	FTより少なくて良い

Parameter-efficient fine-tuning の種類



Parameter-efficient fine-tuning (PEFT)

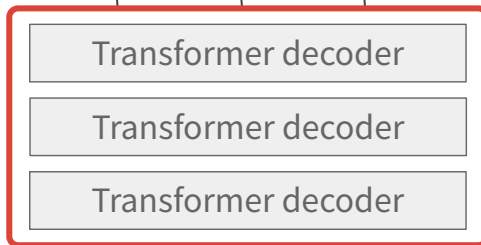
ケース①：soft prompts



学習済みモデルをタスク特化にする full fine-tuning

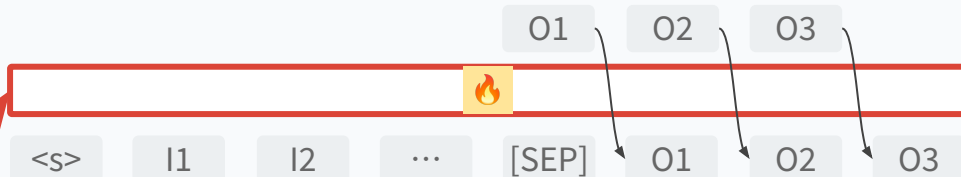
Next token prediction で
事前学習

I eat noodl e ...

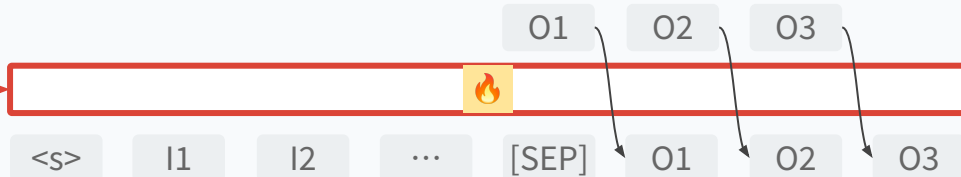


<s> I eat noodl e

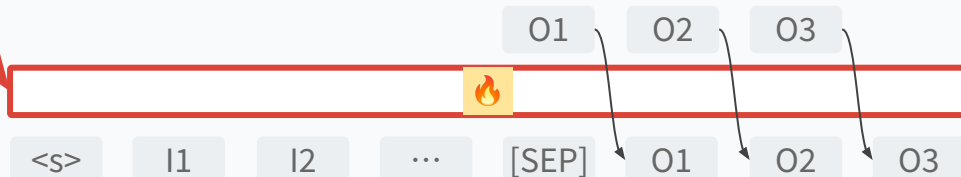
機械翻訳データで学習 (日本語文 I → 英語文 O)



Q&Aデータで学習 (質問文 I → 回答文 O)



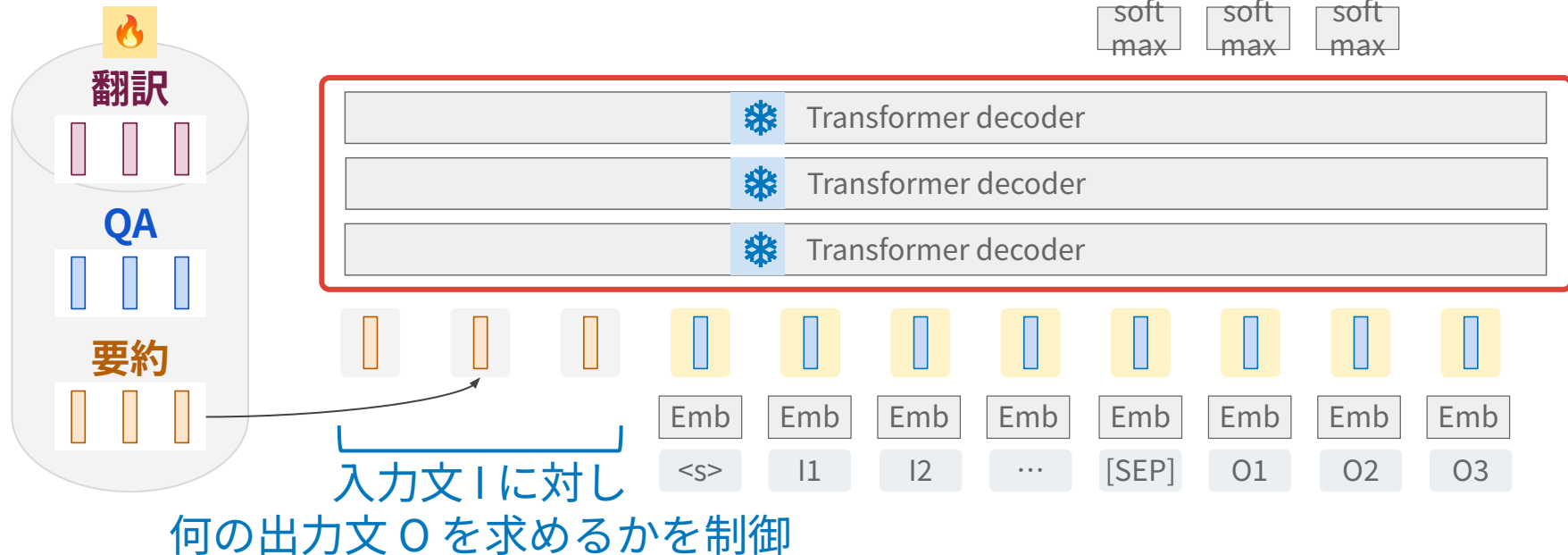
要約データで学習 (元文 I → 要約文 O)



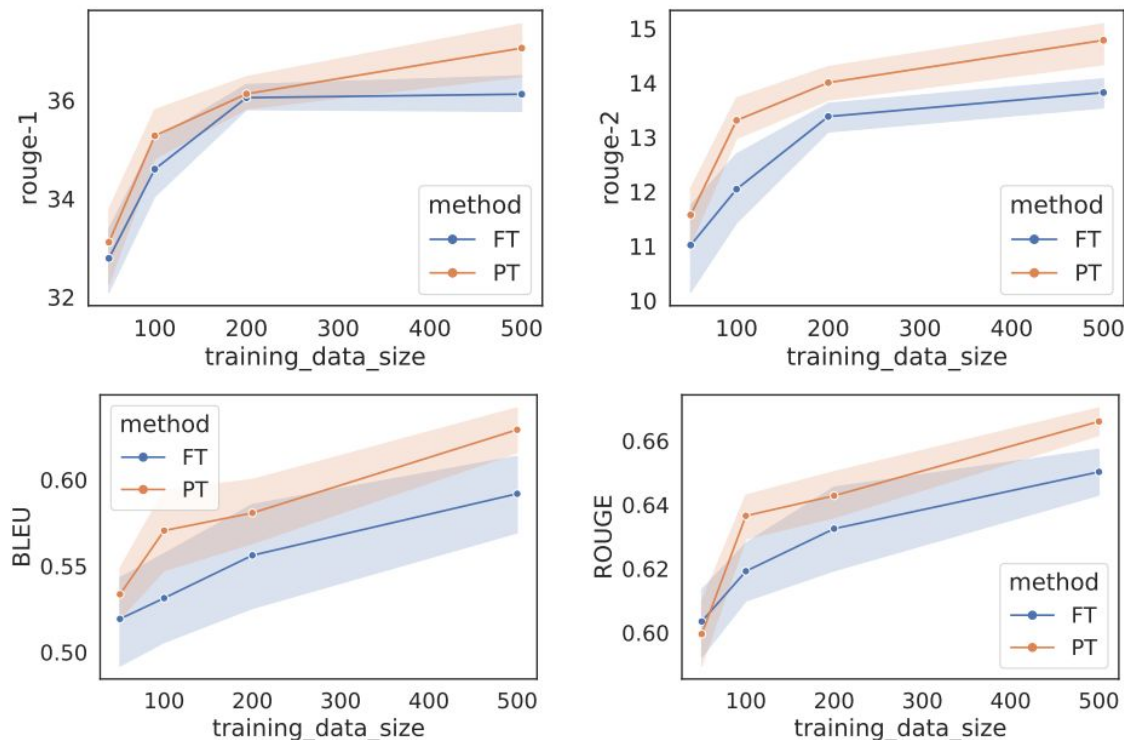
Prefix tuning: タスクを担う task prompt を学習

複数タスクに特化した単一モデルを学習．扱うタスクを task prompt で制御

Task prompt

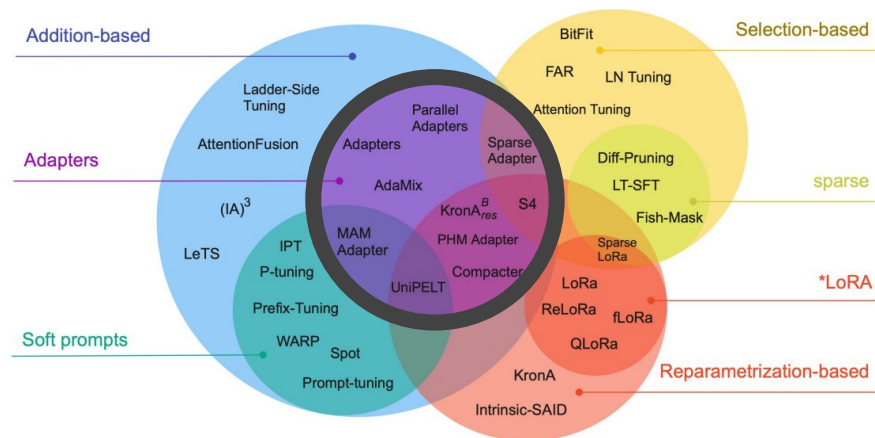


学習データ量が少ないときには full fine-tuning より有利



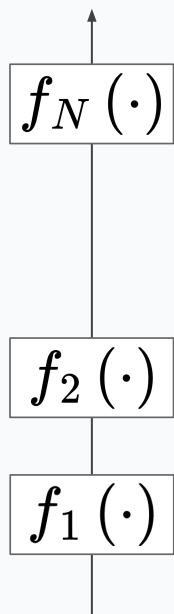
(FT: full fine-tuning, PT: prefix training)

Parameter-efficient fine-tuning (PEFT) ケース②：adapters

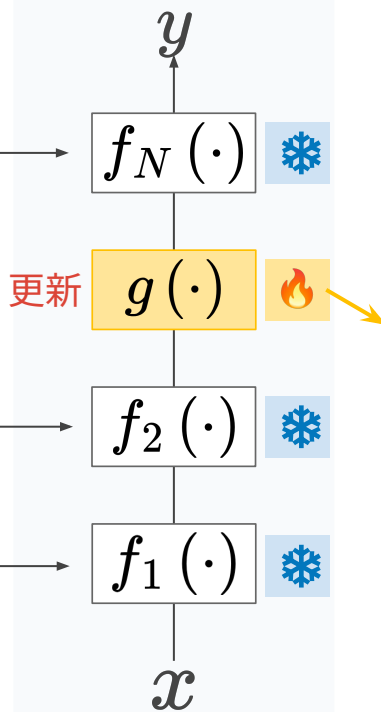


Adapter の考え方

事前学習



w/ adapter



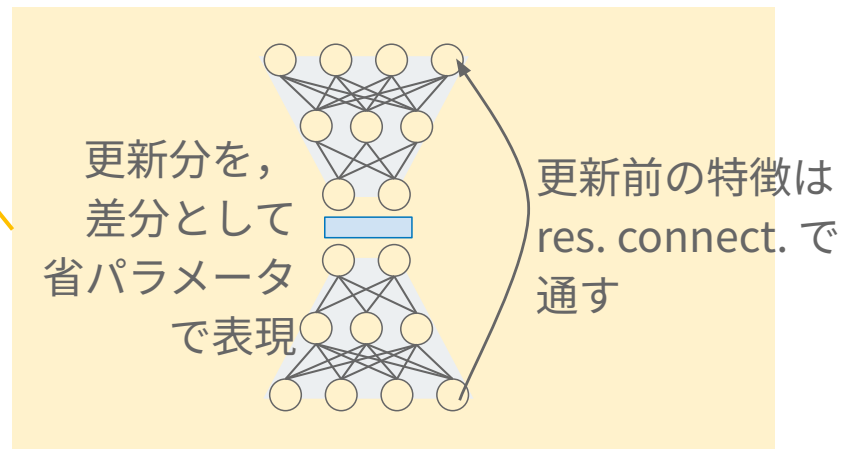
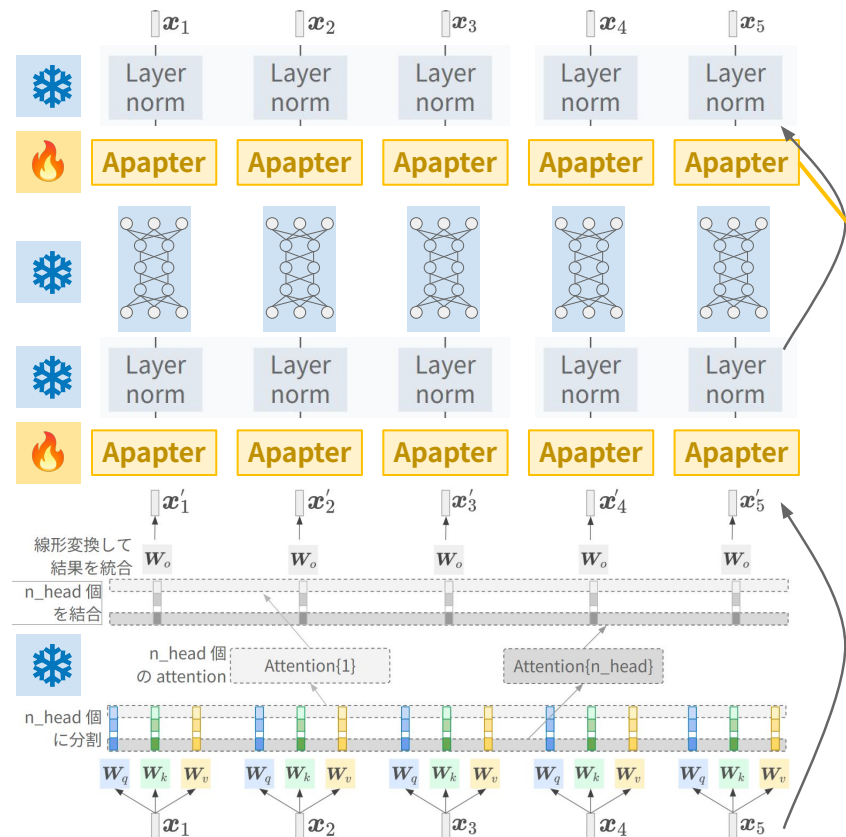
$g()$ が線形変換なら x から y を求めるように
拡大縮小・回転・せん断

下図の1点が x の1点に相当 (例えば単語)



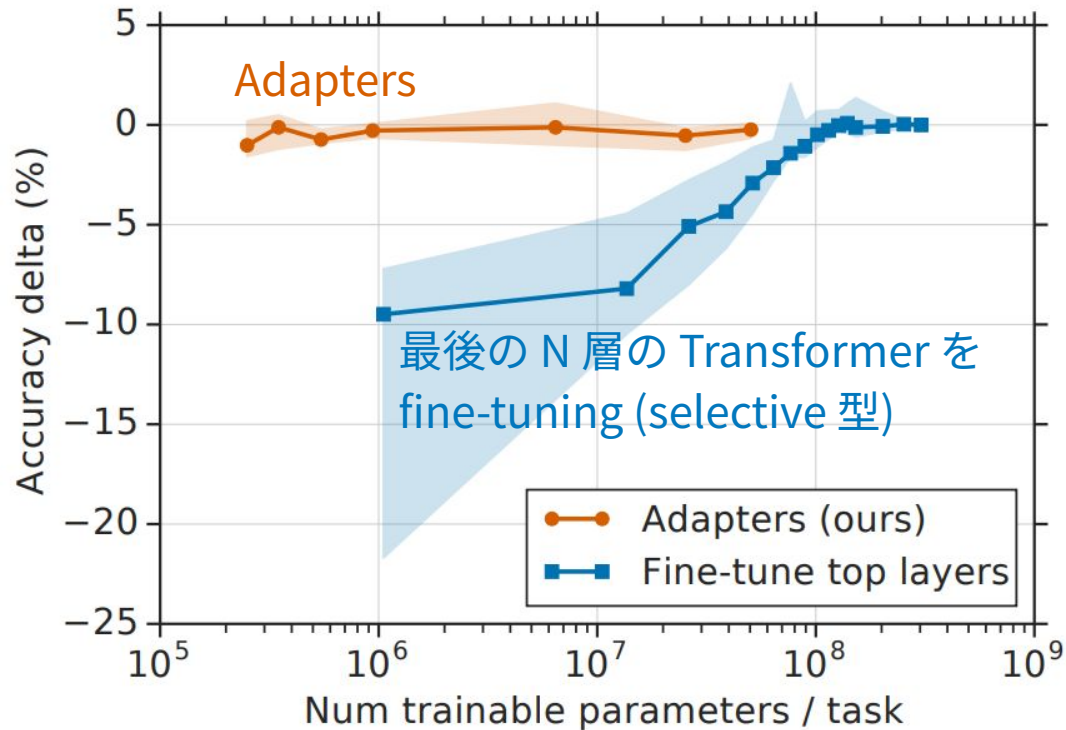
<https://www.ajimatics.com/entry/2018/10/31/060000>

Transformer を使用する場合



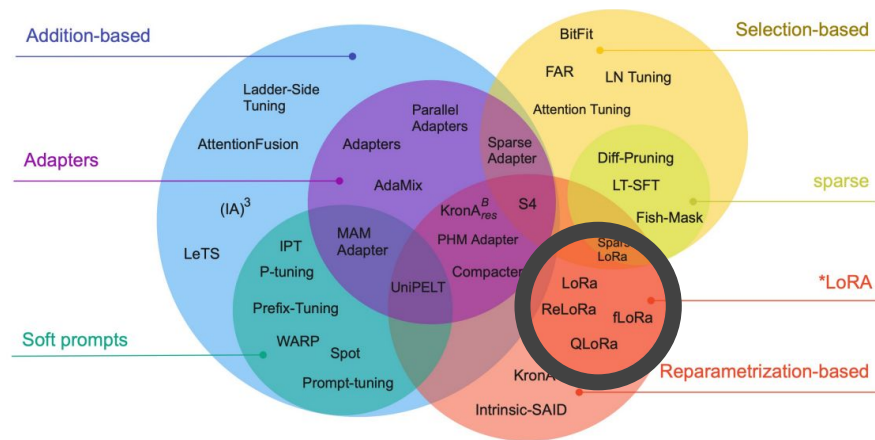
追加位置とアーキテクチャには様々な亜種がある。
例えば Parallel Adapter.

非常に少ないパラメータ数で高い性能を達成



Parameter-efficient fine-tuning (PEFT)

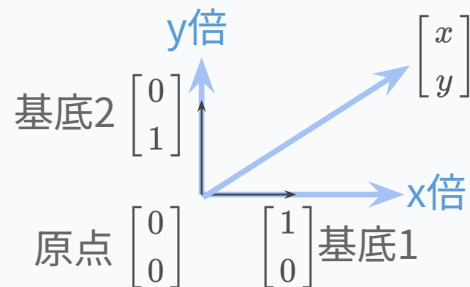
ケース③：LoRA



復習：線形代数・線形変換・アフィン変換

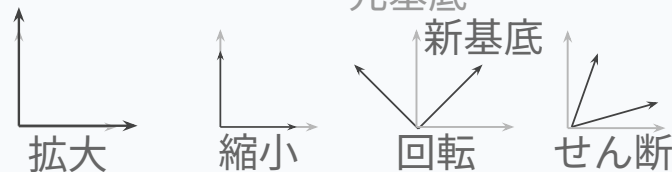
ベクトルは「基底ベクトルを何倍するか」の集まり

$$\begin{bmatrix} x \\ y \end{bmatrix} = x \begin{bmatrix} 1 \\ 0 \end{bmatrix} + y \begin{bmatrix} 0 \\ 1 \end{bmatrix} + \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$



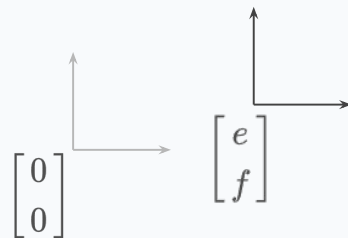
線形変換 $y = Ax$ は基底ベクトルの入れ替え．行列は新たな基底ベクトルの集まり

$$\begin{bmatrix} a & b \\ c & d \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} ax + by \\ cx + dy \end{bmatrix} = x \begin{bmatrix} a \\ c \end{bmatrix} + y \begin{bmatrix} b \\ d \end{bmatrix}$$



アフィン変換 $y = Ax + b$ は基底ベクトルの入れ替え&シフト

$$\begin{bmatrix} a & b \\ c & d \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} + \begin{bmatrix} e \\ f \end{bmatrix} = \begin{bmatrix} ax + by + e \\ cx + dy + f \end{bmatrix} = x \begin{bmatrix} a \\ c \end{bmatrix} + y \begin{bmatrix} b \\ d \end{bmatrix} + \begin{bmatrix} e \\ f \end{bmatrix}$$



復習：行列のランク



逆行列
階数
零空間

行列の低ランク表現 (特異値分解はこの一種)

行列は基底とアクティベーションの積に分解できる

$$\begin{array}{c} \text{基底} \quad \text{アクティベーション} \\ \begin{bmatrix} 1 & 4 & 7 \\ 2 & 5 & 8 \\ 3 & 6 & 9 \end{bmatrix} = \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 \end{bmatrix} + \begin{bmatrix} 4 \\ 5 \\ 6 \end{bmatrix} \begin{bmatrix} 0 & 1 & 0 \end{bmatrix} + \begin{bmatrix} 7 \\ 8 \\ 9 \end{bmatrix} \begin{bmatrix} 0 & 0 & 1 \end{bmatrix} \\ \begin{bmatrix} 1 & 0 & 0 \\ 2 & 0 & 0 \\ 3 & 0 & 0 \end{bmatrix} \quad \begin{bmatrix} 0 & 4 & 0 \\ 0 & 5 & 0 \\ 0 & 6 & 0 \end{bmatrix} \quad \begin{bmatrix} 0 & 0 & 7 \\ 0 & 0 & 8 \\ 0 & 0 & 9 \end{bmatrix} \end{array}$$

低ランク行列は少数の項の和で表現できる

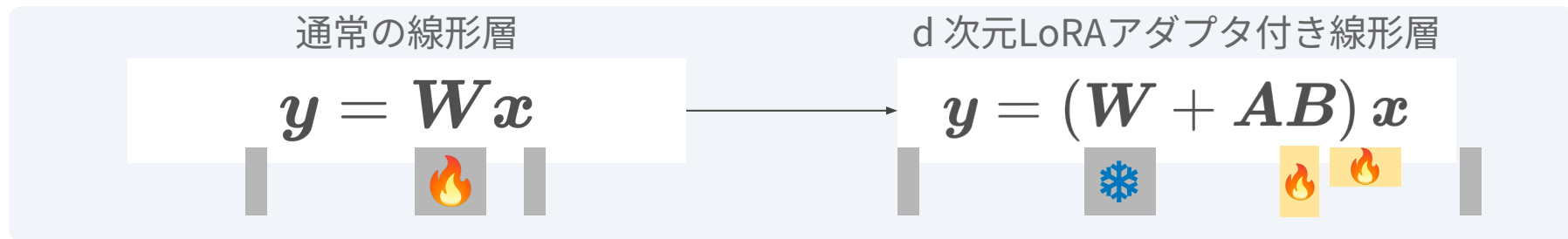
$$\begin{bmatrix} 6 & 3 & 7 \\ 4 & 2 & 8 \\ 2 & 1 & 9 \end{bmatrix} = \begin{bmatrix} 6 \\ 4 \\ 2 \end{bmatrix} \begin{bmatrix} 1 & 0.5 & 0 \end{bmatrix} + \begin{bmatrix} 7 \\ 8 \\ 9 \end{bmatrix} \begin{bmatrix} 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 6 & 7 \\ 4 & 8 \\ 2 & 9 \end{bmatrix} \begin{bmatrix} 1 & 0.5 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

基底がパターン，アクティベーションが基底の出現を表す

$$\begin{bmatrix} 6 & 3 & 0 \\ 4 & 2 & 0 \\ 2 & 1 & 0 \end{bmatrix} \quad \begin{bmatrix} 0 & 0 & 7 \\ 0 & 0 & 8 \\ 0 & 0 & 9 \end{bmatrix}$$

基底 (パターン) の数
= ランク

Low-Rank Adaptation (LoRA)



学習済みの行列に対し、低ランク行列を追加してその行列のみを更新。

$$W^{(\text{new})} \leftarrow W + AB$$

W は正方行列としているが、
正方行列でなくてもよい。

$\mathbb{R}^{N \times N}$

$\mathbb{R}^{N \times N}$

$\mathbb{R}^{N \times d}$ $\mathbb{R}^{d \times N}$



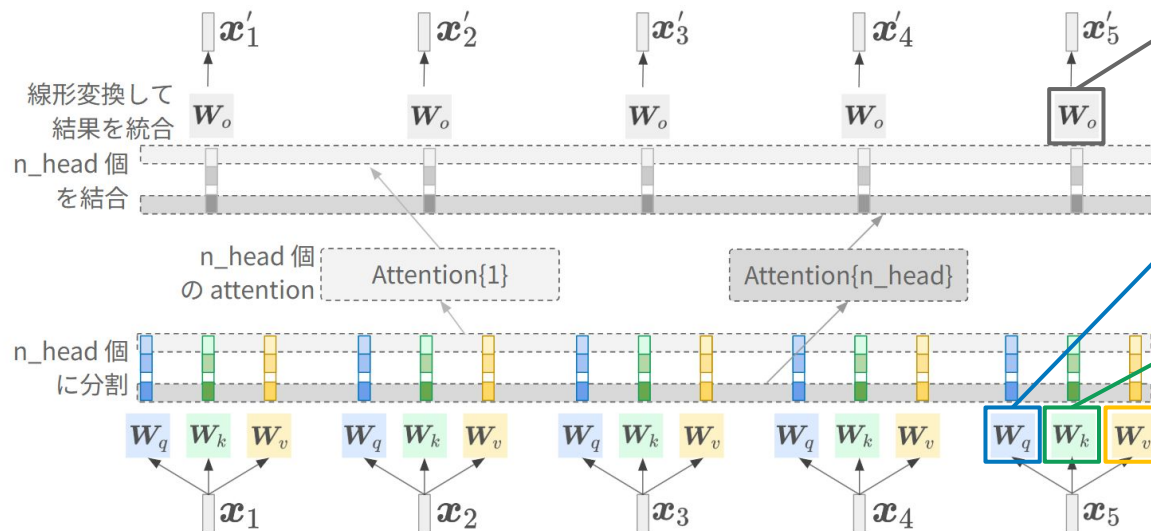
更新しない

$$\left\{ \begin{array}{l} d=1 \\ (1\text{パターン}) \end{array} \right. \begin{bmatrix} 6 \\ 4 \\ 2 \end{bmatrix} \begin{bmatrix} 1 & 0.5 & 0 \end{bmatrix} = \begin{bmatrix} 6 & 3 & 0 \\ 4 & 2 & 0 \\ 2 & 1 & 0 \end{bmatrix}$$
$$\left\{ \begin{array}{l} d=2 \\ (2\text{パターン}) \end{array} \right. \begin{bmatrix} 6 & 7 \\ 4 & 8 \\ 2 & 9 \end{bmatrix} \begin{bmatrix} 1 & 0.5 & 0 \\ 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 6 & 3 & 7 \\ 4 & 2 & 8 \\ 2 & 1 & 9 \end{bmatrix}$$

Self attention に適用する場合

各行列に対し LoRA アダプタ (線形層x2) を追加すればよい

Multi-head 注意機構：q, k, v を分割して注意することで異なる特徴抽出を可能に



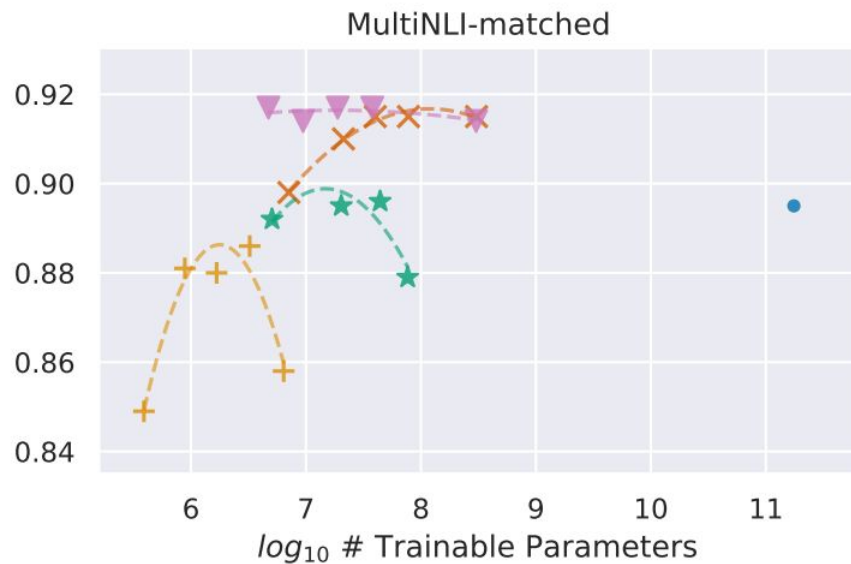
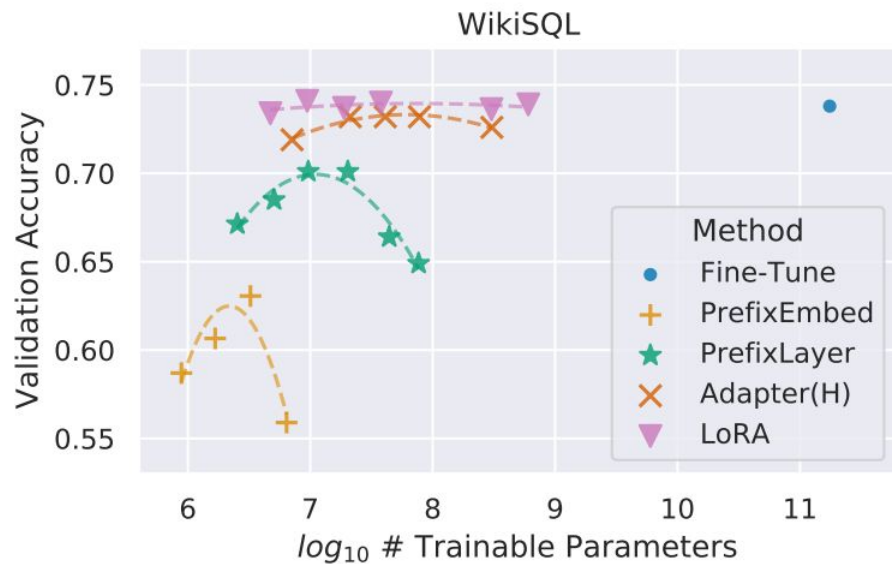
$$W_o \leftarrow W_o + B_o A_o$$

$$W_q \leftarrow W_q + B_q A_q$$

$$W_k \leftarrow W_k + B_k A_k$$

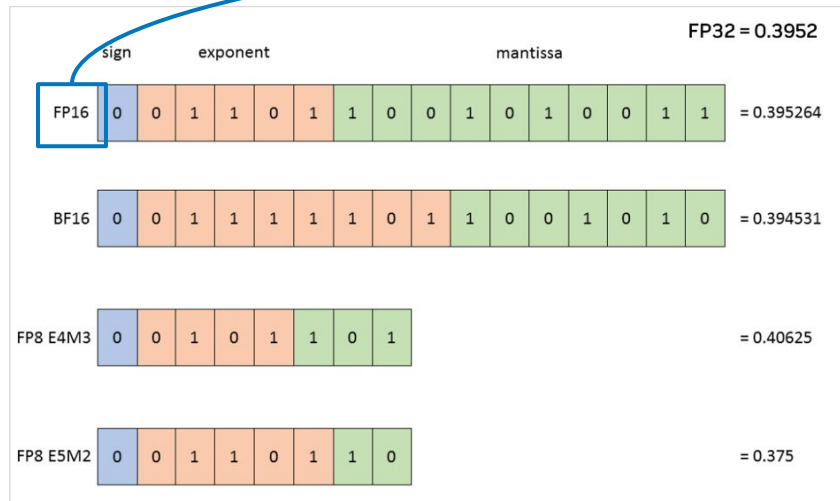
$$W_v \leftarrow W_v + B_v A_v$$

種々のタスクで頑健に動作



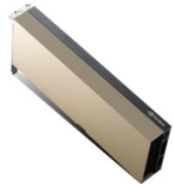
モデルパラメータのデジタル表現とモデル量子化

モデルがメモリに載りきらないことはある．その際にはモデル量子化を行う．
例えば，float16 (FP16) のモデルを float8 (FP8) にすることでメモリは半分になる



$$\text{値} = (-1)^{\text{符号}} \times (1 + \text{仮数}) \times 2^{\text{指数}}$$

要素	計算式	20Bモデルの理論値
モデル	$N_{\text{param}} \times \text{size(float16)}$	40GB
勾配	$N_{\text{param}} \times \text{size(float16)}$	40GB
最適化器 (Adam)	$N_{\text{param}} \times \text{size(float16)} \times 2$	80GB
小計	$N_{\text{param}} \times \text{size(float16)} \times 4$	160GB



NVIDIA H200 Tensor Core GPU
141 GB

画像提供: アブライドネット

NVIDIA
NVIDIA H200 NVL 141GB [PCIExp 141GB]

最安価格
5,305,000円 [価格推移グラフ](#)

メーカー希望小売価格: -円

売れ筋ランキング
👑 92位 (15917製品中)

レビュー
[レビューを書く](#)

クチコミ
[クチコミを書く](#)

お気に入り
[お気に入り](#) (7人)

スペック・仕様 [すべてのスペック・仕様を見る](#)

QLoRA

低量子化 Transformer に対する LoRA. 量子化誤差の小さな量子化と
Out of Memory 回避のための「CPUへの一時的なデータ対比」

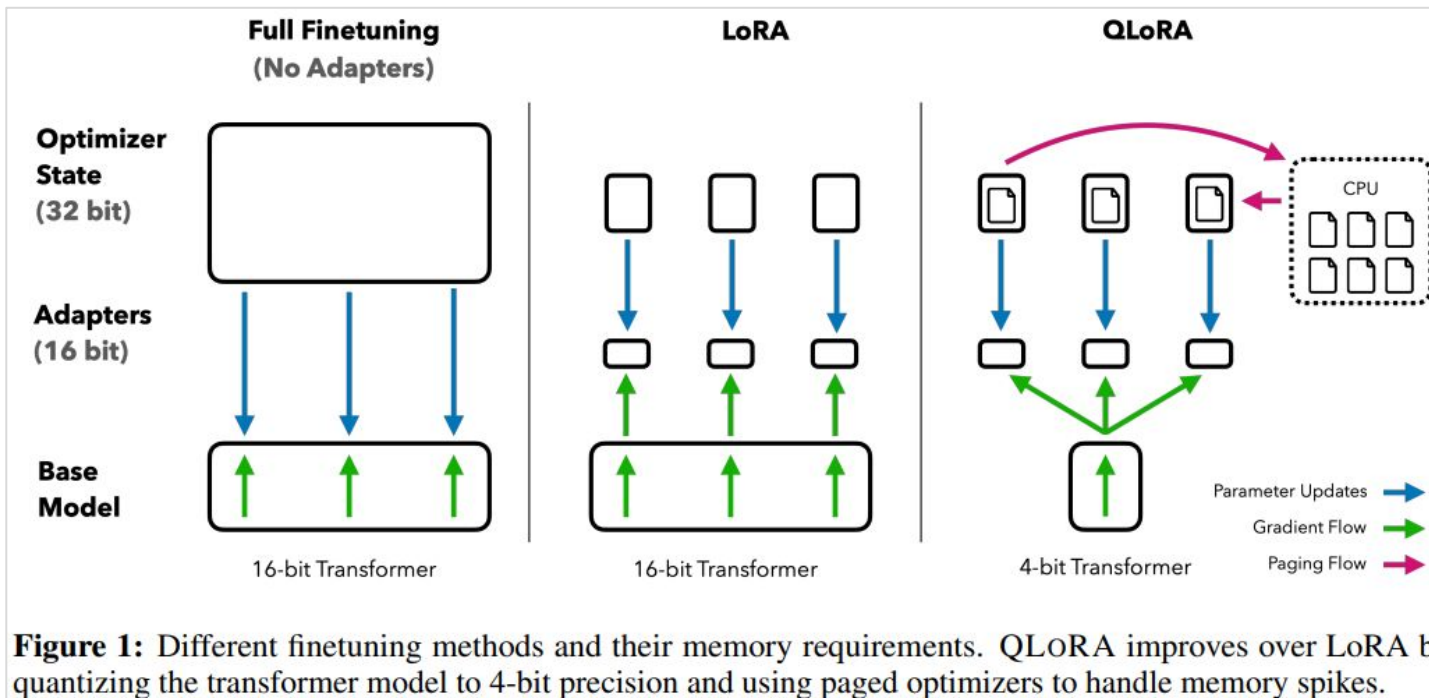


Figure 1: Different finetuning methods and their memory requirements. QLoRA improves over LoRA by quantizing the transformer model to 4-bit precision and using paged optimizers to handle memory spikes.

まとめ

まとめ

- 転移学習の事例
 - 転移学習とは、知識を他タスクに転用すること
 - 色々な LLM がタスク特化型モデルへと転移学習されている
- モデルパラメータ再学習をできない場合がある
 - Parameter-efficient fine-tuning が登場
- Parameter-efficient fine-tuning (PEFT)
 - soft prompts
 - adapters
 - LoRA

課題

なし（講義資料作成で力尽きました…）

興味があれば <https://github.com/huggingface/peft> を動かしてみてください。